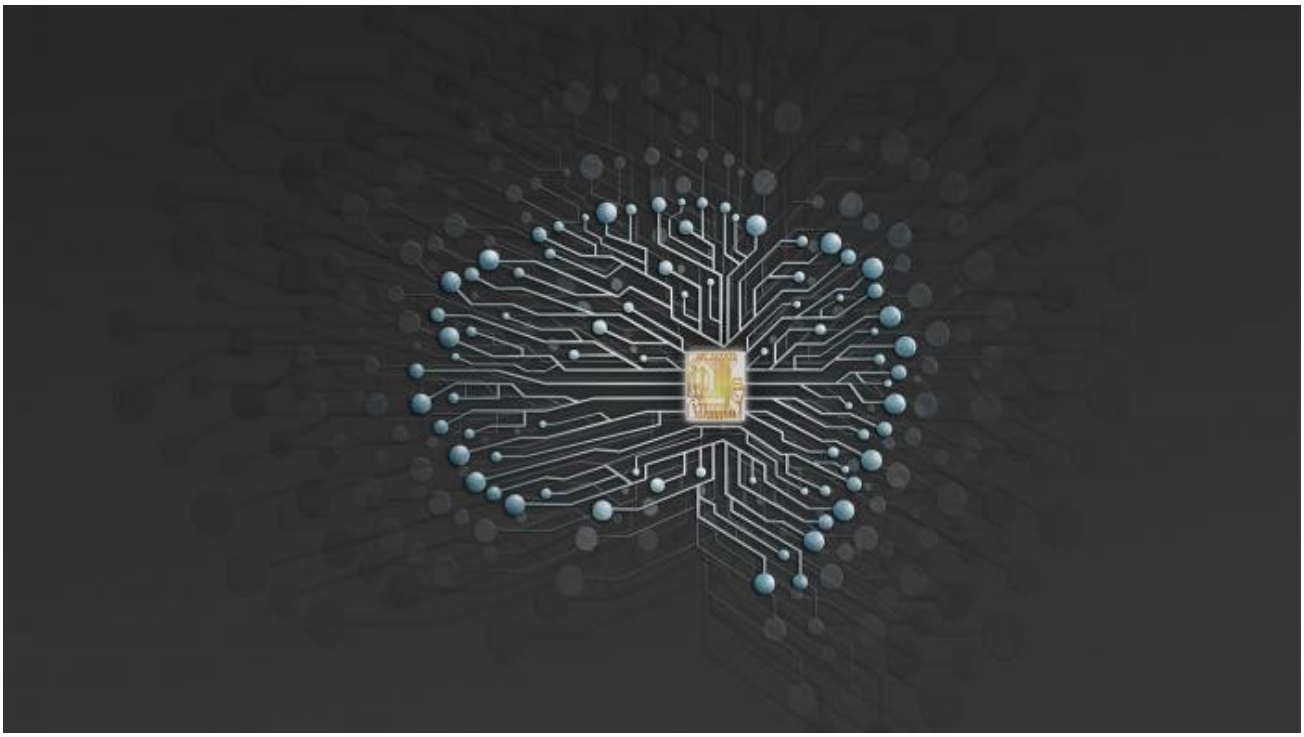
MAGAZIN FÜR PROFESSIONELLE
INFORMATIONSTECHNIK

Deep-Learning-Anwendungen mit Nvidias Jetson Nano

23.01.2020 07:00 Uhr
Ramon Wartala



Einplatinencomputer wie die Jetsons von Nvidia eignen sich gut für eigene Experimente mit Deep Learning, besonders bei Bild- und Objekterkennung sowie Robotik.

Das Paradebeispiel für die Bilderkennung in Echtzeit ist autonomes Fahren. Aktuelle Bilderkennungsalgorithmen, die auf rechenintensiven Deep-Learning-Verfahren basieren, benötigen viel Rechenleistung, um Wegmarken, andere Verkehrsteilnehmer und Hindernisse in Echtzeit erkennen zu können. Eine der größten Herausforderungen bei der Entwicklung elektrisch betriebener autonomer Fahrsysteme ist jedoch die Akkukapazität. Die Leistung eines modernen Akkus wird nicht nur für die Fortbewegung mithilfe des Elektromotors benötigt, sondern auch für die gesamte Bordelektronik inklusive Klima, Navigation und Entertainment. Bei autonom operierenden Fahrzeugen kommt dazu noch die Leistungsaufnahme für die Sensorik und die Verarbeitung des von den Sensoren gelieferten Datenstroms.

Moderne Grafikkarten, wie sie in Serversystemen als Rechenbeschleuniger zum Einsatz kommen, benötigen für diesen Zweck viel zu viel Strom. So nimmt eine High-End-Grafikkarte wie Nvidias Titan RTX rund 280 Watt an einem 650-Watt-Netzteil auf, um die über 4600 CUDA-Kerne anzutreiben. Im Vergleich dazu nimmt sich der "Full Self-driving Computer" (FSD), den Elon Musk 2019 auf den Autonomy Days von Tesla präsentierte, in seiner Leistungsaufnahme von 72 Watt moderat aus. Die zwei sogenannten Neural Network Arrays ermöglichen jeweils rund 72 Billionen (72×10^{12}) Multiplikationen oder Additionen pro Sekunde.

MEHR ZU KI IN UNTERNEHMEN

- **Künstliche Intelligenz unternehmerisch nutzen [1]**
- **Deep-Learning-Anwendungen mit NVIDIAs Jetson Nano [2]**
- **Wie KI bei der Absicherung des IoT helfen kann [3]**

Diese geballte Rechenpower wird zur Bilderkennung in Echtzeit benötigt. Dabei muss das System jede Millisekunde gleich mehrere Bilder, die aus unterschiedlichen Kameras des Fahrzeugs kommen, verarbeiten.

Darf es etwas weniger sein?

Nicht jede ML-Anwendung braucht jedoch so viel Rechenleistung. Wer selbst mit Deep Learning und Bilderkennung experimentieren will, dem bietet Nvidia seit 2014 eine Entwicklungsplattform für autonome Systeme an.

Zur Jetson genannten Familie von Einplatinencomputern kommt jedes Jahr ein weiteres Mitglied hinzu. Auf dem Niveau eines Tesla FSD bewegt sich der Jetson Xavier, der bei einer Leistungsaufnahme von um die 30 Watt bis zu 30 TOPS realisiert.

Nvidias jüngster Sprössling ist der Jetson Xavier NX, der bei 15 Watt Leistungsaufnahme bis zu 21 TOPS an Rechenleistung schafft und ab März 2020 erhältlich sein soll. Genauso kreditkartenklein ist der im Frühjahr 2019 erschienene Jetson Nano. Er wird seinem Namen zweifach gerecht, denn der Nano ist auch das günstigste SoC der Familie. Bei 10 Watt Leistungsaufnahme bringt er es noch auf rund 0,5 TFlops bei Berechnungen mit halber Genauigkeit (FP16).

MASSEINHEITEN FÜR NEURONALE RECHENOPERATIONEN

Geschwindigkeitsangaben können nicht nur bei CPUs zu Verwirrung führen. Die Berechnung von Flops (die Abkürzung steht für Floating Point Operations per Second – Gleitkommaoperationen pro Sekunde), also der Anzahl von Rechenoperationen pro Sekunde, ist bei modernen Prozessorarchitekturen kompliziert. Grund dafür sind verschiedene Turbo-, Hyper-Thread- oder AVX-Erweiterungen, die spezielle Berechnungsansprüche adressieren. Auch die Taktgeschwindigkeit ist nicht immer ausschlaggebend für die Geschwindigkeit in bestimmten Anwendungsbereichen. Moderne CPUs rechnen für gewöhnlich mit Gleitkommazahlen von 32 oder 64 Bit Länge (FP32 oder FP64).

Moderne Frameworks für Deep Learning nutzen in der Regel FP16- anstelle von FP32-Operationen. Das ändert nicht viel an der Genauigkeit der Modelle, verkürzt aber die Trainingszeit signifikant. Etabliert hat sich die Maßeinheit TFlops, also Teraflops, was 10^{12} Gleitkommaoperationen pro Sekunde entspricht.

Möchte man mithilfe von Deep-Learning-Modellen Bild- und Objekterkennung in Echtzeit, zum Beispiel über eine angeschlossene Kamera, implementieren, bietet der Nano dafür genug Leistungsreserven. Die Tabelle gibt einen Überblick über die Leistungswerte der Jetson-Familie. Allen Jetson-Systemen gemein ist das ARM-basierte System on a Chip (SoC) Tegra. Im Nano sind 4 CPU-Kerne bei 64 Bit enthalten. Softwareseitig ist ein angepasstes Ubuntu Linux 18.04 installiert.

Leistungswerte der Jetson-Familie

Name	Jahr	Stromverbrauch	GPU	TFlops (FP16)	Preis
TK1	2014	~10 Watt	192 Core Kepler	0,3	~170 EUR
TX1	2015	~10–1 Watt	256 Core Maxwell	1,0	~270 EUR
TX2	2017	~7,5–15 Watt	256 Core Pascal	1,3	~350 EUR
Xavier	2018	~10–30 Watt	512 Core Volta	11,0	~870 EUR
Nano	2019	~5–10 Watt	128 Core Maxwell	0,5	~90 EUR
	2020	~10–15 Watt		6,0	

Xavier NX	384 Core Volta	399 US-$ (ab März 2020)

Trainieren und schlussfolgern

Das Training komplexer und tiefer neuronaler Netze erfordert viel Speicher und noch mehr Prozessorleistung. Um eine hohe Modellqualität zu erreichen, sind mitunter sehr viele Trainingsdaten und viele Berechnungsiterationen nötig. Wer also glaubt, er könne eine moderne Grafikkarte für das Modelltraining ohne Weiteres durch ein System on a Chip ersetzen, irrt leider. Diese Systeme nutzen normalerweise extern – auf einem leistungsfähigeren System – trainierte Modelle, die im Anschluss zum Beispiel zur Erkennung von Objekten in Echtzeit zum Einsatz kommen.

Doch nicht nur das Training benötigt ordentlich Rechenpower. Sobald man ein vortrainiertes ML-Modell auf reale Daten anwendet – für dieses "Schlussfolgern" hat sich der Begriff Inference durchgesetzt –, muss das Modell im Speicher liegen. Die zu analysierenden Daten müssen zudem durch alle Operatoren des Netzes durchgeleitet werden, um zum Beispiel ein Objekt auf einem Bild mit einer der trainierten Klassen zu identifizieren. Solche Modelle können auch gern mehrere MByte groß werden und mehrere Hunderttausend oder Millionen Gewichte enthalten, die es bei der Klassifizierung zu berücksichtigen gilt.

[4]

So mancher Entwickler von autonomen oder Embedded-Systemen wünscht sich dabei die Einfachheit eines Raspberry Pi in Kombination mit einer ausgewachsenen GPU. Diese

Zielgruppe scheint Nvidia mit seinem Jetson Nano ansprechen zu wollen. Seit Mitte 2019 vertreibt das Unternehmen den Jetson Nano auch in Deutschland.

Vom Einplatinen-Urgestein Raspberry Pi hat der Nano auch seine Anschlussmöglichkeiten über GPIO-Kontaktstifte (General Purpose Input/Output) geerbt. Neben einer vollwertigen Ethernet-Schnittstelle, einem microSD-Kartenslot und vier USB-Anschlüssen besitzt der Nano einen HDMI-2.0-Ausgang mit Full-HD-Unterstützung sowie den vom Raspberry Pi gewohnten CSI Camera Connector, an den man eine bereits vorhandene Raspi-Kamera anschließen kann.





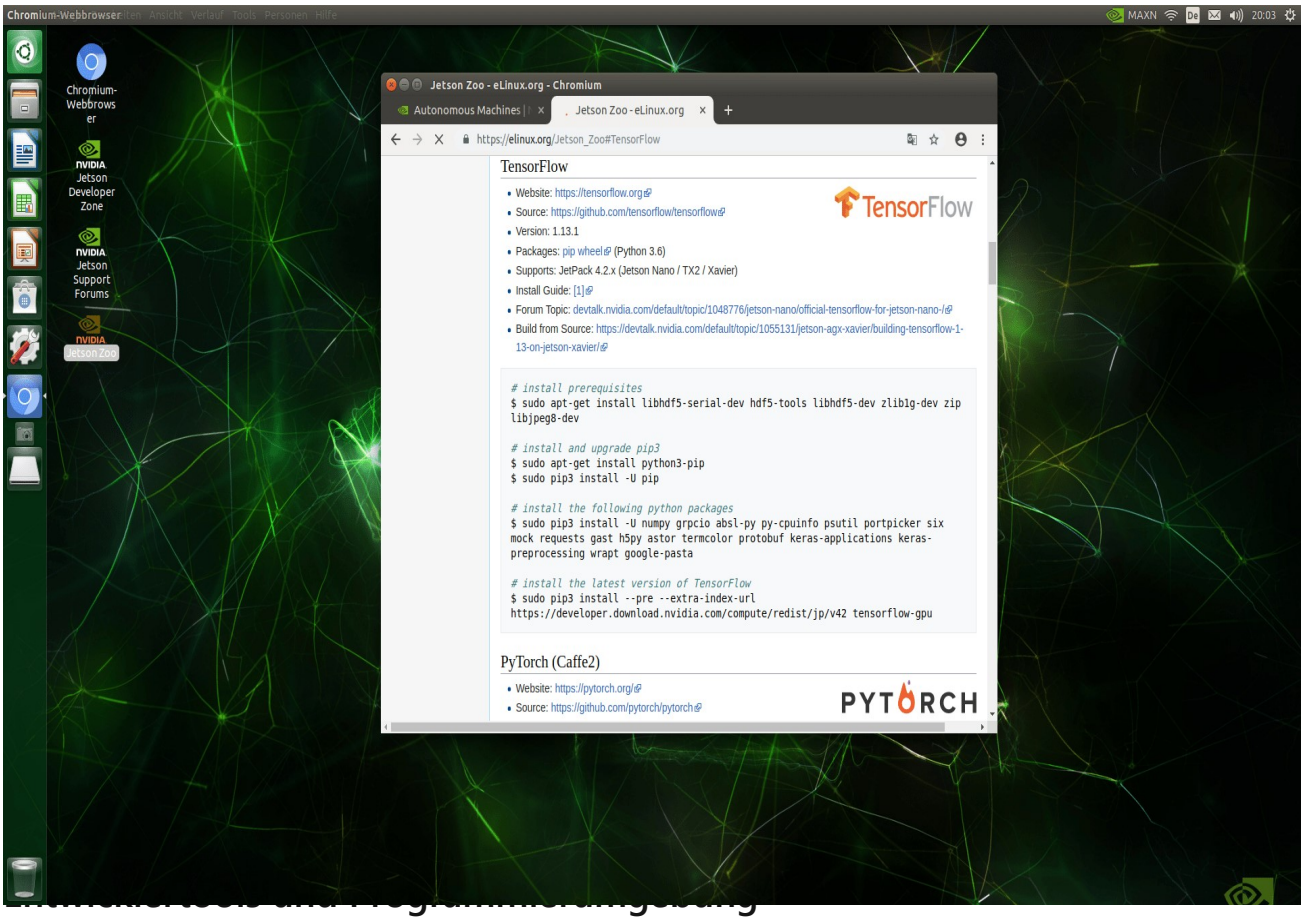
Vertrauens zu erwerben. Nvidia liefert das System ohne Netzteil, Tastatur und Maus, microSD-Karte oder andere Hardware. Wer allerdings die Infrastruktur für einen Raspberry Pi besitzt, hat in der Regel neben einem WLAN-USB-Dongle auch ein Kameramodul in der Version 2 oder höher.

Ist die Hardware beschafft, benötigt der Jetson Nano ein Boot-Image, das man von der Nvidia-Website herunterladen und gemäß der dort ebenfalls zu findenden Anleitung installieren kann. Die microSD-Karte für das Image sollte ausreichend groß dimensioniert sein und für ernsthafte Experimente mindestens 32 GByte umfassen.

Das Boot-Image enthält unter anderem das SDK JetPack in der jeweils aktuellen Version und ein speziell angepasstes Linux für Nvidias auf ARM64 basierendes Ein-Chip-System Tegra mitsamt allen Bibliotheken und APIs sowie diversen Development-Tools, Beispielen und Dokumentationen.

JetPack läuft auf allen Mitgliedern der Jetson-Familie, sodass ein späteres Upgrade auf ein leistungsfähigeres System praktischerweise ohne großen Aufwand möglich ist. Auf Nvidias Downloadseite finden sich auch alle Links zur Dokumentation der verschiedenen Systeme wie des Driver Package für Linux (L4T), der Multimedia API, cuDNN, TensorRT (siehe Abschnitt "Modellbeschleunigung mit TensorRT") und Nvidias DeepStreaming SDK.

Nach dem Booten präsentiert sich dem Benutzer die aufgeräumte grafische Benutzeroberfläche eines abgespeckten Ubuntu Linux in Full-HD. Möchte man den Jetson Nano später "headless" nutzen, ist dies seit Version 4.2.2 auch möglich.



Entwicklertools und Programmierumgebung

Auf den ersten Blick enthält der Jetson Nano alle Tools, die man auf einem Linux-System erwarten würde. Bei Bedarf lässt sich Software über den grafischen Installer oder über den Paketmanager apt nachinstallieren.

Für die **Bilderkennung ist die Open Source Computer Vision Library** [5], kurz OpenCV, bereits vorinstalliert. Diese eignet sich besonders gut, um eine angeschlossene Kamera zu testen. Ein einfaches Beispiel inklusive Gesichtserkennung findet sich im GitHub-Repository der JetsonHacks. CSI Camera erkennt Gesichter in Echtzeit, wenn sie von der Kamera erfasst werden. Folgende Zeilen in der Shell starten die Anwendung:

```
git clone https://github.com/JetsonHacksNano/ CSI-Camera.git
cd CSI-Camera
python3 face_detect.py
```

Um mehr als Gesichter zu erkennen, muss man andere Werkzeuge und Frameworks bemühen. Sogenannte Convolutional Neural Networks (CNNs) sind hier der Stand der Technik. Diese Art künstlicher neuronaler Netze besteht vereinfachend gesagt aus vielen Schichten, durch die ein Bild oder Tensor (zum Beispiel Höhe × Breite × Anzahl Farbkanäle) so lange geschoben wird, bis das Netz darin enthaltene Objekte aus der Ausgangsschicht klassifizieren kann. Sowohl die Architektur besonders erfolgreicher CNNs als auch bereits

trainierte Modelle sind im Rahmen der gängigen Deep-Learning-Frameworks wie PyTorch, Caffe, Keras, MXNet oder TensorFlow verfügbar. Sie bestehen in der Regel aus Zigtausenden von trainierten Gewichten, die gern mehrere Megabyte groß werden können.

Modellbeschleunigung mit TensorRT

GPU-seitig nutzt Nvidia die Eigenentwicklung TensorRT. Es handelt sich dabei um eine hochoptimierte Laufzeitumgebung zur Verarbeitung von Operationen auf Tensoren auf den eignen Grafikprozessoren. TensorRT ist spezialisiert auf Modelloptimierung und auf Inference. Die Runtime basiert auf der ebenfalls von Nvidia entwickelten Compute Unified Device Architecture (CUDA) sowie auf der darauf aufbauenden Deep-Learning-Bibliothek CuDNN.

Je nach Zielsystem bietet TensorRT unterschiedliche Optimierungen an. Auf dem Jetson Nano unterstützt die Software die hardwarenahe Beschleunigung von Modellen mit geringer Präzision mit INT8- und FP16-Werten.

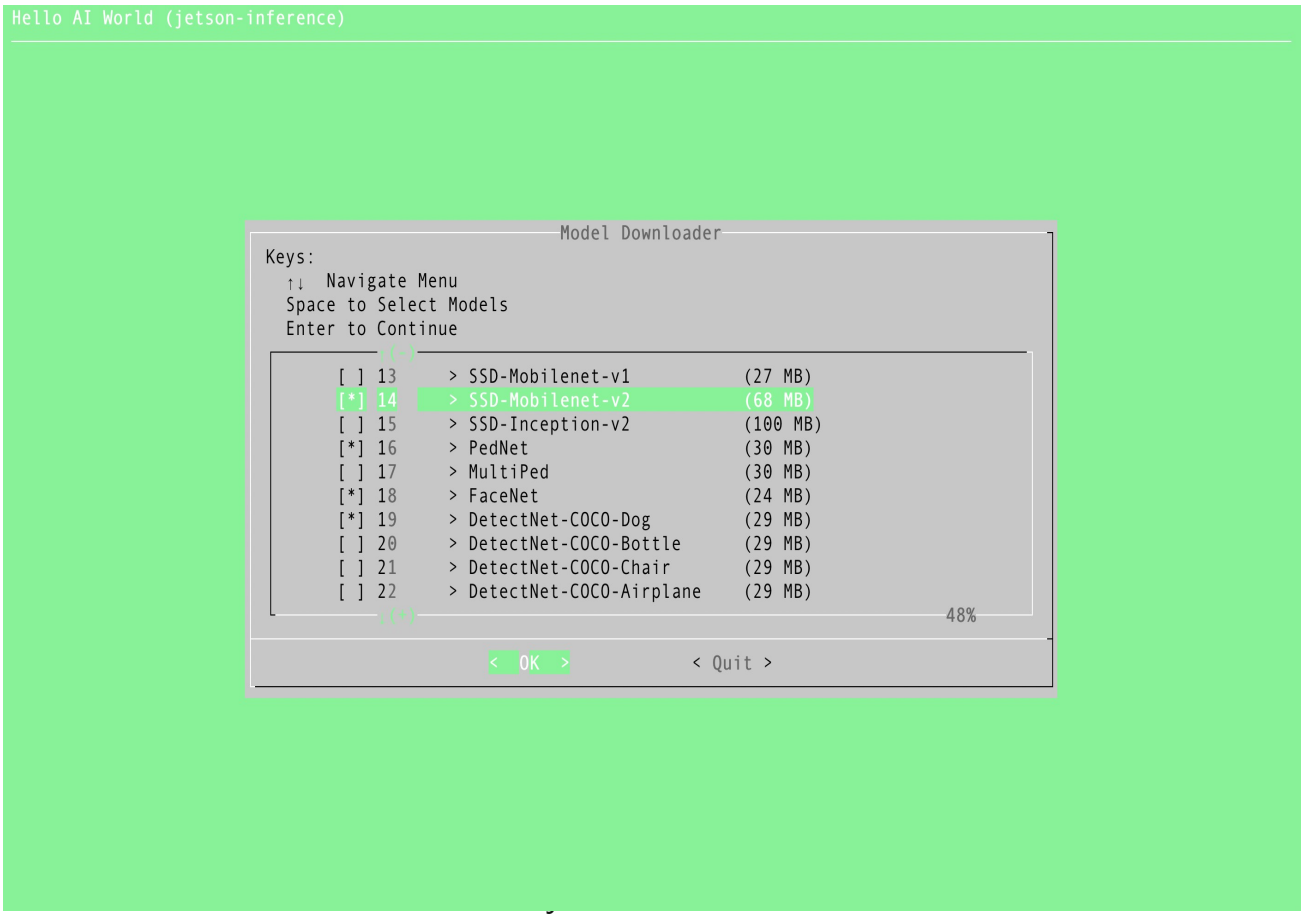
Einen guten Ausgangspunkt für eigene Experimente mit TensorRT bietet Dustin Franklins Projekt **jetson-inference auf GitHub [6]**. Das Repository ist mit

```
git clone https://github.com/dusty-nv/ jetson-inference.git
```

schnell auf dem Jetson Nano ausgecheckt und mit den folgenden Befehlen übersetzt und installiert.

```
cd jetson-inference
git submodule update --init
mkdir build
cd build
cmake ..
make
sudo make install
```

Bei der Installation bietet der Model Downloader eine große Auswahl an vortrainierten Deep-Learning-Modellen zum Herunterladen an. Darunter befinden sich bekannte wie das AlexNet von 2012 sowie verschiedene sogenannte Residual Neural Networks (ResNets) für die Bilderkennung. Ebenfalls dabei sind SSD-Mobilenet-v2, das 90 Objekte (von Apfel bis Zahnbürste) erkennt, und DeepScene der Universität Freiburg zur Erkennung von Objekten im Straßenverkehr (Verkehrsschilder, Fußgänger, Motorräder, Häuser …).



Beispielanwendungen. Die Tabelle gibt einen Überblick über die enthaltenen Python-Beispiele.

Die wichtigsten Python-Beispielprogramme im jetson-inference-Repository

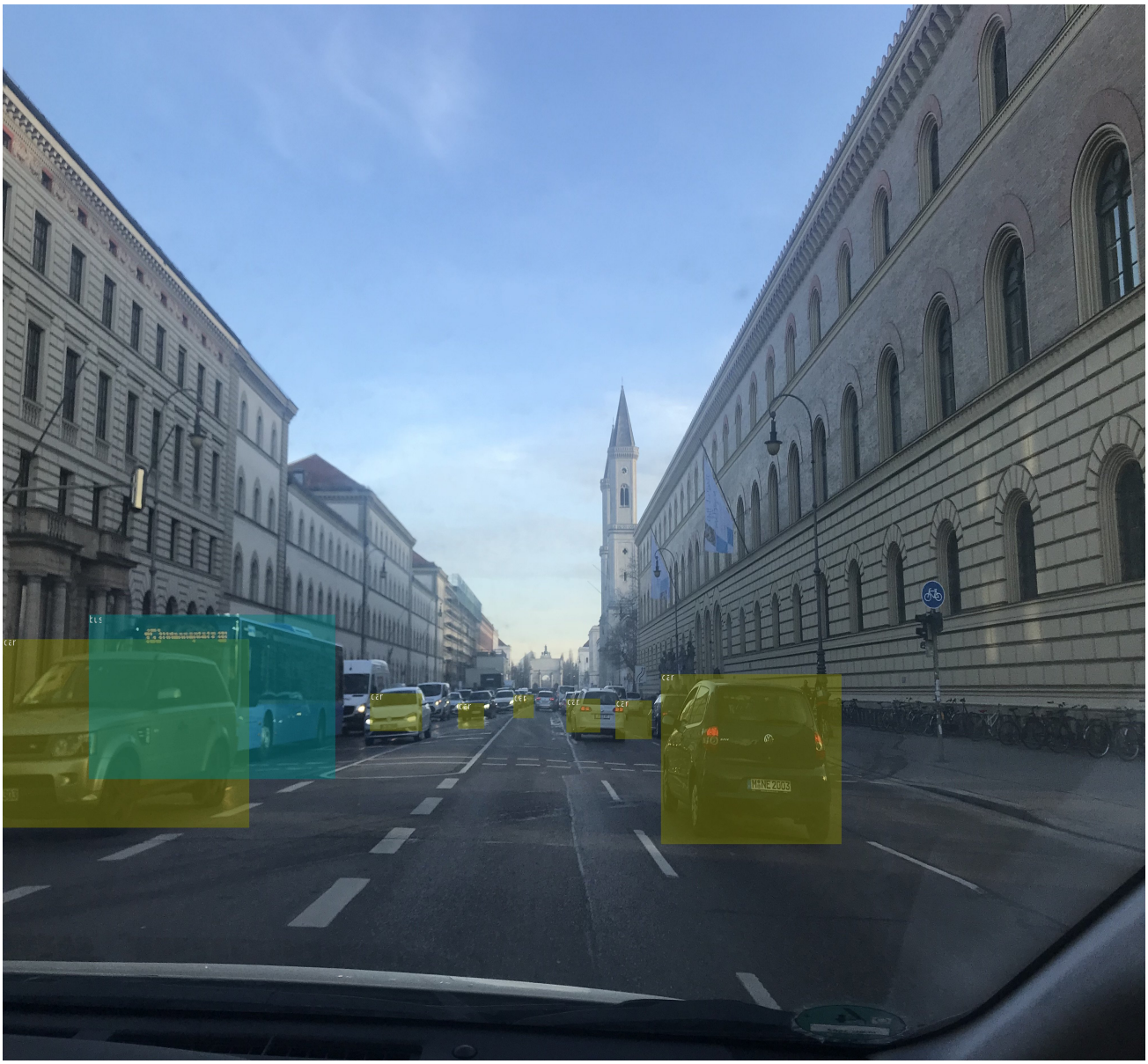
Name	Beschreibung
imagenet-console.py	dateibasierter Bildklassifikator
detectnet-console.py	lokalisiert Objekte in einem Bild
segnet-console.py	segmentiert Objekte in Bildern
imagenet-camera.py	Bildklassifikation mit der Kamera
detectnet-camera.py	Objektlokalisierung im Kamerabild
segnet-camera.py	Objektsegmentierung im Kamerabild

Ohne Kamera lassen sich am einfachsten eigene Bilder mit dem Nano verarbeiten. Der Aufruf

```
python3 imagenet-console.py --network=googlenet ~/Bilder/flugzeug.jpg output.jpg
```

klassifiziert mithilfe des GoogleNet-Modells das Objekt auf einem Foto vom Gate am Züricher Flughafen mit 0,916217-prozentiger Wahrscheinlichkeit als ‚Airliner'.

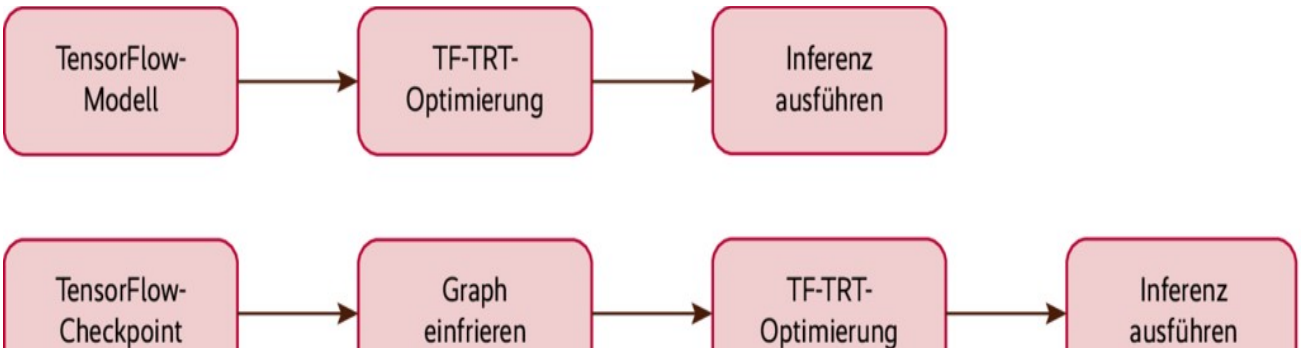




Framework integriert sein.

TensorRT und TensorFlow

Seit Mai 2018 ist TensorRT vollständig in das von Google entwickelten Machine-Learning-Framework TensorFlow integriert, sodass man keine zusätzlichen Werkzeuge bemühen muss. Ein TensorRT-optimiertes Modell lässt sich sowohl aus einem gespeicherten als auch aus einem "eingefrorenen" TensorFlow-Modell erzeugen.





Modell direkt nach TensorRT oder man friert es auf einem bestehenden Trainings-Checkpoint ein und wandelt es dann in ein TensorRT-optimiertes Modell um. Letzteres ist sinnvoll, wenn man Modelle aus einem laufenden Training benötigt.

Auf dem so gespeicherten Modell führt TensorRT folgende Operationen aus: Zuerst scant und partitioniert es den TensorFlow-Graphen und optimiert diesen mithilfe der eingebauten GPU-optimierten Operatoren. Dann konvertiert es den TensorFlow-Layer in für TensorRT optimierte Layer. Zuletzt konvertiert es die partitionierten Graphen in die jeweils spezifische GPU-Variante von TensorRT. TensorRT-optimierte Modelle sind in der Regel rund achtmal schneller, als wenn man sie in TensorFlow selbst ausführen würde.

Die aktuelle TensorFlow-Version lässt sich über die Kommandozeile mit folgenden Zeilen auf dem Nano installieren:

```
sudo pip3 install --pre --extra-index-url https://developer.download.nvidia.com/
compute/redist/jp/v42 tensorflow-gpu
```

Für die Modellerstellung für TensorRT kann man auch den kostenlosen Dienst Colab von Google nutzen. Der Vorteil dieser Herangehensweise ist, dass man sowohl eine GPU- als auch eine TPU-Instanz nutzen kann und dass TensorFlow bereits integriert ist.

Bestehende Modelle für TensorRT aufbereiten

Im **GitHub-Repository zu diesem Artikel [7]** findet sich das Jupyter-Notebook keras_model_to_tensorrt.ipyn. Es implementiert einen Beispiel-Konverter, der ein bestehendes, mithilfe der Deep-Learning-Bibliothek Keras entwickeltes Modell in TensorRT umwandelt.

Der Konverter lädt das erstellte Modell mit den Anweisungen

```
from tensorflow.keras.applications. mobilenet_v2 import MobileNetV2 as mn
...
keras_model = mn(weights='imagenet')
```

herunter und speichert es lokal mit

```
keras_model.save(keras_model_filename)
```

Das erste Listing friert das Modell ein, während das zweite es anschließend als TensorFlow-Modell abspeichert. Der Abschnitt im dritten Listing konvertiert es und speichert es in ein TensorRT-Modell im Protobuf-Format ab.

```
def freeze_graph(graph, session, output, save_pb_dir='.',
save_pb_name='frozen_model.pb', save_pb_as_text=False):
    with graph.as_default():
        graphdef_inf = tf.graph_util.remove_training_nodes(graph.as_graph_def())
        graphdef_frozen = tf.graph_util.convert_variables_to_constants(session,
graphdef_inf, output)
        graph_io.write_graph(graphdef_frozen, save_pb_dir, save_pb_name,
as_text=save_pb_as_text)
        return graphdef_frozen
```

```
...
keras_model = load_model(keras_model_filename)
...
frozen_graph = freeze_graph(
    session.graph,
    session,
    [out.op.name for out in keras_model.outputs],
    save_pb_dir=save_pb_dir
)
```

```
...
trt_graph = trt.create_inference_graph(
    input_graph_def=frozen_graph,
    outputs=output_names,
    max_batch_size=1,
    max_workspace_size_bytes=1 << 25,
    precision_mode='FP16',
    minimum_segment_size=50
)

graph_io.write_graph(trt_graph, save_pb_dir, trt_model_filename, as_text=False)
```

Das so erzeugte Modell lässt sich aus Google Colab heraus mit dem Codeausschnitt aus dem folgenden Listing auf den Jetson Nano herunterladen.

```
...
from google.colab import files
files.download(save_pb_dir+trt_model_filename)
```

Im selben Repository findet sich unter dem Namen tensorrt-keras-prediction.py ein vollständiges Python-Beispiel, das ein so gespeichertes TensorRT-Modell zur Bildklassifikation nutzt. Mit dem folgenden Code lädt man zunächst das gespeicherte Modell und holt dann den Eingabe- und Ausgabe-Tensor des Modells holt..

```
...
def get_frozen_graph(graph_file):
    with tf.io.gfile.GFile(graph_file, "rb") as f:
        graph_def = tf.compat.v1.GraphDef()
        graph_def.ParseFromString(f.read())
    return graph_def

trt_graph = get_frozen_graph('./models/mobilenetv2-model-trt.pb')
```

```
output_names = ['Logits/Softmax']
input_names = ['input_1']
input_tensor_name = input_names[0] + ":0"
output_tensor_name = output_names[0] + ":0"

output_tensor = tf_sess.graph.get_tensor_by_name(output_tensor_name)
```

Für die Bildklassifizierung wird das Bild über die Keras-Hilfsfunktion ```tensorflow.keras.preprocessing``` geladen und in die TensorFlow-Session injiziert. Danach wird das Ergebnis ausgegeben.

```
from PIL import Image
from tensorflow.keras.preprocessing import image
img = image.load_img(img_path, target_size=image_size[:2])
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

feed_dict = {
```

```
    input_tensor_name: x
}
preds = tf_sess.run(output_tensor, feed_dict)

print('Predicted:', decode_predictions(preds, top=3)[0])
```

Das Flugzeug vom Züricher Flughafen wird nun richtig als

`Predicted: [('n02690373', 'airliner', 0.70644724), ('n02687172', 'aircraft_carrier', 0.15092887), ('n04552348', 'warplane', 0.052062042)]`

erkannt.

Robotik und Rennwagen

Weitere Anwendungsbeispiele für den Jetson Nano **finden sich auch auf der Community-Webseite von Nvidia [8]**. Für die Analyse von innerstädtischem Straßenverkehr gibt es mit **Open DataCam [9]** ein DIY-Projekt, das das leistungsfähige **YOLO-Modell [10]** für die Echtzeiterkennung nutzt.

Neben der reinen Bildverarbeitung gibt es noch weitere interessante Anwendungsfelder. So bietet der **JetBot [11]** einen preiswerten Einstieg in die autonome Robotik. Etwas weniger beschaulich geht es mit dem **JetRacer [12]** zu, der einen Nano-basierten autonomen Rennwagen implementiert. **Das Projekt ‚Hey, Jetson!' [13]** nutzt den Jetson Nano zur Spracherkennung, während das Jupyter-Notebook charRNN mit TensorRT ein zeichenbasiertes Sprachmodell mithilfe eines Recurrent Neural Network (RNN) optimiert.

Fazit

Der Jetson Nano stellt einen guten Kompromiss zwischen kostengünstiger Hardware, Leistungsaufnahme und Rechenpower dar. Er ist besonders für Entwickler und Hobbyisten interessant, die schon erste Programmiererfahrungen mit Systemen wie dem Raspberry Pi gesammelt haben und auf der Suche nach geeigneter Hardware für Deep-Learning-Experimente sind. Die Jetson-Plattform ist gut in Nvidias sonstige Grafikhardwareprodukte integriert, sodass sich Projekte auch skalieren lassen. Besonders für Bild- und Objekterkennung sowie für Robotik-Experimente eignet sich der Nano – Kenntnisse in Linux, C++ und Python vorausgesetzt.

Wer sich jetzt fragt, ob sich damit nicht auch wunderbar Kryptowährungen minen lassen, der muss jetzt ganz stark sein: ASICs (Application Specific Integrated Circuits) haben GPUs bei der Berechnung von Bitcoin- oder Ethereum-Hashes schon X-fach überflügelt.

Ramon Wartala ist Director Data Science bei SinnerSchrader (Part of Accenture Interactive) in Hamburg und berät mit seinem Team Kunden in Datenarchitekturen für Maschine-Learning-Anwendungen. Ende 2017 erschien sein Buch "Praxiseinstieg Deep Learning" im O'Reilly Verlag.

LESEN SIE AUCH

Wie KI bei der Absicherung des IoT helfen kann

heise+

[14]

(akl [15])

URL dieses Artikels:

https://www.heise.de/-4640511

Links in diesem Artikel:

[1] https://www.heise.de/hintergrund/Kuenstliche-Intelligenz-unternehmerisch-nutzen-4640432.html

[2] https://www.heise.de/ratgeber/Deep-Learning-Anwendungen-mit-NVIDIAs-Jetson-Nano-4640511.html

[3] https://www.heise.de/ratgeber/Wie-KI-bei-der-Absicherung-des-IoT-helfen-kann-4640688.html

[4] https://www.heise.de/ix/

[5] https://www.heise.de/select/ix/2017/11/1509393829637286

[6] https://github.com/dusty-nv/jetson-inference

[7] https://github.com/rawar/jetson-nano-obj-detect

[8] https://developer.nvidia.com/embedded/community/jetson-projects

[9] https://github.com/opendatacam/opendatacam

[10] https://pjreddie.com/darknet/yolo/

[11] https://github.com/NVIDIA-AI-IOT/jetbot

[12] https://github.com/NVIDIA-AI-IOT/jetracer

[13] https://github.com/bricewalker/Hey-Jetson

[14] https://www.heise.de/ratgeber/Wie-KI-bei-der-Absicherung-des-IoT-helfen-kann-4640688.html

[15] mailto:akl@ix.de

Copyright © 2020 Heise Medien